On-Policy and Off-Policy Temporal-Difference Control for Warehouse Robot Navigation

A controlled comparison of Q-learning and SARSA,
implemented from first principles and evaluated over thirty paired seeds

---

Case Study Report

Reinforcement Learning

Smart Warehouse Robot Navigation — learning the shortest collision-free delivery route

---

Research question	How do Q-learning and SARSA compare when a robot must learn the shortest collision-free route to a delivery point?
Dataset	`zoya77/robot-path-planning-navigation-dataset` 7,980 warehouse snapshots; 5,045 distinct 5×5 layouts
Algorithms	Q-learning (off-policy) and SARSA (on-policy), both written from scratch in NumPy
Protocol	30 paired seeds per algorithm, identical budget, Wilcoxon signed-rank on the paired differences

Source code and executed notebook:
`https://github.com/Ashwin-3cS/transformer-t1`

October 31, 2026

Abstract

This report presents a controlled empirical comparison of the two classical tabular temporal-difference control algorithms — Q-learning, which is off-policy, and SARSA, which is on-policy — on the problem of learning a collision-free delivery route across a warehouse floor. Both algorithms are implemented from first principles, sharing every component except the single expression that forms their update target, so that the comparison isolates the on-policy/off-policy distinction rather than any difference of implementation.

Obstacle layouts are taken from a published Kaggle dataset of warehouse navigation grids. A column by column audit of that dataset is reported as part of the methodology, because several of its columns prove to carry no usable information: its reward signal correlates -0.03 with the true shortest-path distance, its path-length field correlates $+0.01$, its convergence flag is constant, and its episode identifiers do not describe coherent trajectories. Those fields are excluded with the evidence shown. What remains sound — 5,045 distinct occupancy grids with valid start and goal cells — is used to build the environment, and exact ground truth is obtained by breadth-first search computed independently.

The report is explicit about the boundary between data and assumption. The racking layout is real. The reward structure, the cross-aisle and the pedestrian walkway that constitutes the hazard are declared modelling choices, and the hazard penalty is subsequently varied across a sweep because the principal finding depends on it.

Thirteen numerical checks verify the environment and both agents before any experiment is run, including an equivalence test establishing that at $\varepsilon = 0$ the two algorithms produce bit-identical action-value tables, as the mathematics requires.

The experiment uses thirty seeds per algorithm, paired so that both algorithms see identical conditions, and reports Wilcoxon signed-rank tests with bootstrap confidence intervals. The results reproduce the classical prediction with large and highly significant effects. Q-learning converges on the optimal 19-step route running directly alongside the walkway, achieving a mean deployed route of 19.40 steps, but accumulates 21,626 safety incidents during training. SARSA learns a route averaging 33.34 steps at more than five times the distance from the walkway, and accumulates 5,068 incidents — a factor of 4.3 fewer — with significantly better online return ($p = 1.9 \times 10^{-9}$).

Two further results qualify the finding. A sweep of the hazard penalty shows Q-learning's behaviour to be completely invariant to it, while SARSA's clearance from the walkway rises monotonically as the penalty grows: the mechanism is visible directly in the measurements. And on a floor without a localised hazard, both algorithms converge on the identical optimum, confirming that the distinction is a response to a property of the task rather than a general ranking of the two algorithms.

Keywords: reinforcement learning, temporal-difference control, Q-learning, SARSA, on-policy

and off-policy learning, warehouse robotics, safe exploration, Cliff Walking.

Contents

Abstract **ii**

1 Introduction **1**

1.1 Background 1
1.2 Problem statement 1
1.3 Research question 2
1.4 Objectives 2
1.5 Scope and delimitations 2
1.6 Contributions 3
1.7 Organisation 3

2 Literature Review **4**

2.1 Overview 4
2.2 Markov decision processes and dynamic programming 4
2.3 Monte Carlo and temporal-difference learning 4
2.4 Q-learning 5
2.5 SARSA 5
2.6 The Cliff Walking result 5
2.7 Related developments 6
2.8 Reinforcement learning for warehouse robots 6
2.9 Research gap 6

3 Theoretical Background **7**

3.1 Markov decision processes 7
3.1.1 Returns and the discount factor 7
3.2 Value functions and the Bellman equations 7
3.3 Temporal-difference learning 8
3.4 The two control algorithms 8
3.4.1 SARSA — on-policy control 8
3.4.2 Q-learning — off-policy control 8

3.4.3 The consequence, stated precisely . . . 9
3.5 The algorithms in full . . . 9
3.6 Exploration and the epsilon schedule . . . 10
3.7 Convergence conditions . . . 10
3.8 Complexity . . . 11

4 Dataset and Environment **12**
4.1 What a dataset can and cannot supply . . . 12
4.2 The dataset . . . 12
4.3 Audit . . . 13
4.3.1 The episode structure is not real . . . 13
4.3.2 The supplied reward signal is noise . . . 13
4.3.3 What is sound . . . 13
4.3.4 Recovering the action encoding . . . 14
4.4 Constructing the floor . . . 14
4.4.1 Why the raw grids are insufficient . . . 14
4.4.2 The floor used . . . 14
4.5 The Markov decision process . . . 14

5 Implementation **17**
5.1 The from-scratch constraint . . . 17
5.2 The environment . . . 17
5.3 The shared agent . . . 18
5.4 The one-line difference . . . 18
5.5 The episode loop . . . 18
5.6 Experimental harness . . . 19

6 Verification **20**
6.1 Why this chapter precedes the results . . . 20
6.2 Environment checks . . . 20
6.3 Ground-truth checks . . . 20
6.4 The decisive check . . . 21
6.5 Learning checks . . . 21
6.6 Results . . . 21
6.7 What this does not establish . . . 21

7 Results **23**
7.1 Headline comparison . . . 23
7.2 Statistical significance . . . 23

7.3 The learned routes . . . 24
7.4 External baselines . . . 25
7.5 Generality across floors . . . 25
7.6 Sensitivity to the hazard penalty . . . 26

8 Discussion **29**
8.1 Answering the research question . . . 29
8.1.1 They are not better or worse; they answer different questions . . . 29
8.1.2 The mechanism, confirmed directly . . . 29
8.1.3 The effect is a response to risk, not a property of the algorithms . . . 29
8.1.4 Which to use, and when . . . 30
8.2 An uncomfortable comparison . . . 30
8.3 Threats to validity . . . 30
8.3.1 Internal validity . . . 30
8.3.2 External validity . . . 31
8.3.3 Construct validity . . . 31

9 Conclusion and Future Work **32**
9.1 Summary . . . 32
9.2 Findings . . . 32
9.3 Future work . . . 32
9.4 Closing remarks . . . 33

References **34**

A Reproduction **35**
A.1 Environment . . . 35
A.2 Execution . . . 35
A.3 Determinism . . . 35

B Source Listings **36**
B.1 The two agents . . . 36
B.2 Floor construction . . . 37

C Verification Output **38**

D Per-Seed Results **39**

E Summary of Key Figures **41**

List of Figures

4.1 The warehouse floor 15

7.1 Deployed routes 24
7.2 Learning curves 25
7.3 State values 26
7.4 Paired differences 26
7.5 Penalty sensitivity 28

List of Tables

4.1 Separation of data from assumption 12
4.2 Correlation of the supplied numeric columns with the true shortest-path distance, computed by breadth-first search. 13
4.3 Properties of the usable portion of the dataset. 13
4.4 The reward structure 16

5.1 Hyperparameters 19

6.1 Checks on the environment. 20
6.2 Verification results 21

7.1 Headline results 23
7.2 Paired significance tests 24
7.3 Baseline comparison 25
7.4 Generality check 27
7.5 Penalty sensitivity 27

E.1 Every headline quantity reported in this study. 41
E.2 Shared experimental quantities. 41

Chapter 1

Introduction

1.1 Background

A modern fulfilment warehouse is a shared workspace. Automated mobile robots move stock between racking and packing stations while people walk the same floor, and the robot's routing problem is therefore not only about finding the shortest path. It is about finding the shortest path that is also safe to execute, and about doing so without causing harm during the period in which it is still learning.

That last qualification is easy to overlook and is the subject of this report. A policy learned in a simulator can be evaluated purely on its final quality, because mistakes made during training cost nothing. A policy learned on a live warehouse floor cannot: every exploratory action is taken in the real world, and the cost of learning is a cost the operator actually pays.

Reinforcement learning offers two classical algorithms for this class of problem, and they differ precisely on this point. Q-learning [13, 14] learns the value of the optimal policy regardless of how the agent is behaving. SARSA [7, 10] learns the value of the policy the agent is actually following, exploration included. On a benign problem the two converge to the same answer. On a problem where exploration can be costly they do not, and the difference is both predictable and practically important.

1.2 Problem statement

The task is navigation on a warehouse floor represented as a grid. A robot starts at a charging station and must reach a delivery point. Racking blocks movement. A pedestrian walkway crosses the floor, and entering it is a safety incident.

The shortest route runs along the main aisle, which lies directly alongside the walkway. A second route one row further from the walkway is slightly longer and carries no such exposure. The problem is interesting because those two routes differ in exactly the way that distinguishes the two algorithms: the fast route is optimal for an agent that never makes mistakes, and dangerous for one that is still exploring.

1.3 Research question

How do Q-learning and SARSA compare when a warehouse robot must learn the shortest collision-free route to a delivery point?

Four subsidiary questions are answered empirically:

1. **Deployed quality.** After training, which algorithm's greedy policy produces the shorter route, and how close is it to the true optimum?
2. **Cost of learning.** Which algorithm accumulates fewer safety incidents while learning, and what does training cost in accumulated reward?
3. **Significance.** Are the observed differences distinguishable from seed-to-seed noise?
4. **Mechanism.** Whatever is observed, what property of the algorithms and of the task accounts for it?

1.4 Objectives

1. Implement a grid-world warehouse environment from first principles, with no reinforcement learning library.
2. Implement Q-learning and SARSA from first principles, sharing all machinery except the update target, so that the comparison isolates the on-policy/off-policy distinction.
3. Audit the source dataset column by column and use only what carries signal, documenting the exclusions with evidence.
4. Verify the environment and both agents numerically before any experiment is run.
5. Compare the algorithms over thirty paired seeds with a paired non-parametric significance test and reported effect sizes.
6. Test the robustness of the result against an external baseline, against independently generated floors, and against the main modelling assumption.

1.5 Scope and delimitations

Tabular methods only. The state space is 400 cells, small enough to enumerate. Nothing here concerns function approximation or deep reinforcement learning, where off-policy bootstrapping raises stability issues that do not arise in the tabular case.

A simulated environment built on real layouts. This point is developed at length in Chapter 4 because it determines what the study can honestly claim. The obstacle layouts come from the dataset; the dynamics and the reward structure are written here.

One task family. All results concern grid navigation with a localised hazard. The report is explicit about which findings depend on that structure.

No tuning contest. Both algorithms share one hyperparameter setting. The objective is a controlled comparison, not the best achievable performance from either.

1.6 Contributions

1. **A documented dataset audit** showing that several columns of a published Kaggle dataset — its reward signal, its path lengths, its episode structure — carry no usable information, with the evidence rather than the assertion.
2. **A verification suite for tabular RL**, including an equivalence test that pins the two algorithms to each other under a greedy policy, where they are mathematically identical.
3. **A properly paired multi-seed comparison**, with significance testing and effect sizes, rather than a single run per algorithm.
4. **An explicit separation of what is data from what is assumed**, together with a sensitivity analysis over the assumption that drives the result.
5. **A negative control**: the demonstration that on a floor without a localised hazard, the two algorithms converge on the identical optimum and the much-cited distinction between them simply does not appear.

1.7 Organisation

Chapter 2 reviews temporal-difference control and the on-policy/off-policy distinction. Chapter 3 develops the mathematics: Markov decision processes, the Bellman equations, TD(0), the two control algorithms and their convergence conditions. Chapter 4 audits the dataset and specifies the environment, separating data from assumption. Chapter 5 presents the implementation. Chapter 6 presents the verification suite. Chapter 7 reports the experiment. Chapter 8 interprets it and states the threats to validity. Chapter 9 concludes.

Chapter 2

Literature Review

2.1 Overview

This chapter traces the development of value-based reinforcement learning up to the two algorithms compared here, and then examines what is and is not well established about the difference between them. The organising question throughout is: *whose* value is the agent learning?

2.2 Markov decision processes and dynamic programming

The formal setting is the Markov decision process, developed by Bellman [1] and given its modern treatment by Puterman [6]. An MDP specifies states, actions, transition probabilities and rewards, and the Markov property asserts that the next state depends only on the current state and action.

Dynamic programming — policy iteration and value iteration — solves an MDP exactly, but requires the transition and reward functions to be known in advance. A warehouse robot does not have them. It has only the ability to act and observe what follows, which is the setting reinforcement learning addresses.

2.3 Monte Carlo and temporal-difference learning

Two families of method learn from experience alone. Monte Carlo methods wait until an episode terminates and update toward the observed return; they are unbiased but high-variance, and cannot learn from incomplete episodes.

Temporal-difference learning, introduced by Sutton [9], instead updates toward a *bootstrapped* estimate — the immediate reward plus the discounted value of the successor state. TD methods learn online, from incomplete episodes, with far lower variance. The cost is bias, since the target depends on the current estimate, which is itself wrong. Convergence results for TD(0) under suitable step-size conditions are given by Dayan [2] and Jaakkola et al. [5].

The practical importance of this trade-off is that TD control learns *while the robot is still moving*, which is what makes the cost of exploration a measurable quantity in this study.

2.4 Q-learning

Q-learning, introduced by Watkins [13] and proved convergent by Watkins and Dayan [14], is the canonical *off-policy* TD control algorithm. Its update bootstraps from the maximum over next-state action values,

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\Big],$$

and this maximisation is what makes it off-policy: the target describes the greedy policy, irrespective of the policy that generated the data. Given sufficient exploration and a suitable step-size schedule, Q converges to the optimal action-value function Q^* regardless of how the agent behaves while learning.

That independence is the algorithm's defining strength and, in the setting studied here, the source of its characteristic weakness: because the target never refers to the behaviour policy, the cost of the agent's own exploration never enters its estimates.

2.5 SARSA

SARSA was introduced by Rummery and Niranjan [7] and named by Sutton [10] after the quintuple (s, a, r, s', a') its update consumes:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big[r + \gamma Q(s',a') - Q(s,a)\Big], \qquad a' \sim \pi(\cdot \mid s').$$

Here a' is the action the agent will actually take. The update therefore evaluates the behaviour policy itself, making SARSA *on-policy*. Under an ε-greedy policy, SARSA converges to the optimal ε-greedy policy rather than to the optimal policy, and converges to Q^* only if ε is decayed to zero appropriately — the GLIE conditions [8].

This apparent weakness is the source of its practical strength. By evaluating the policy that includes exploration, SARSA accounts for the possibility that the next action is random, and therefore assigns lower value to states from which a random action would be catastrophic.

2.6 The Cliff Walking result

The clearest demonstration of the distinction is Example 6.6 of Sutton and Barto [11], *Cliff Walking*. An agent crosses a grid from start to goal; a row of cliff cells lies between them. Stepping into the cliff incurs a large penalty and returns the agent to the start.

The optimal path runs immediately alongside the cliff. Q-learning learns exactly that path, and consequently falls off repeatedly while exploring, so its online performance during learning is poor. SARSA learns a longer path further from the edge, because its estimates price in the possibility of an ε-greedy step over it, and achieves better online return. Decaying ε makes both converge to the optimal policy.

Three features of this result are essential and are reproduced deliberately in the present study:

- The hazard is **localised and adjacent to the optimal path**. Without this there is no decision to make and the two algorithms behave identically.

- The environment is **deterministic**. The effect is caused by ε-greedy exploration alone, not by stochastic dynamics.
- The comparison measures **two distinct quantities** — online return during learning, and the quality of the final greedy policy — which have different answers.

The third point is the one most often lost in secondary accounts, which tend to report that "SARSA is safer" without noting that Q-learning's final policy is the better one.

2.7 Related developments

Expected SARSA [12] replaces the sampled $Q(s', a')$ with its expectation under the policy, reducing variance while remaining on-policy; it subsumes both algorithms as special cases. Double Q-learning [4] addresses the maximisation bias that the max operator introduces. Neither is evaluated here, since the object of study is the basic on-policy/off-policy contrast, but both are natural extensions (Chapter 9).

The distinction acquires additional importance under function approximation, where off-policy bootstrapping combined with approximation and bootstrapping forms the "deadly triad" [11] and can diverge outright. In the tabular setting studied here both algorithms are convergent, so the difference observed is about the *policy being evaluated* and not about stability.

2.8 Reinforcement learning for warehouse robots

Automated guided vehicles and autonomous mobile robots in warehouses are a well-established application area, and safety around human co-workers is a recognised constraint rather than an afterthought. The field of safe reinforcement learning, surveyed by García and Fernández [3], is explicitly concerned with the problem this report measures: the cost incurred *during* learning, as distinct from the quality of the policy that results.

Within that framing, the on-policy/off-policy choice is one of the simplest available mechanisms for trading final optimality against safety while learning, and it requires no additional machinery — only a different bootstrap target.

2.9 Research gap

The Cliff Walking result is textbook material, usually presented as a single illustrative run on a 4×12 grid. What is less commonly available is that demonstration carried out with the methodological care that would be expected of an empirical claim: multiple seeds, paired statistical testing, an external optimality reference, a check that the effect generalises across problem instances, and a sensitivity analysis over the hazard penalty that produces it.

This report supplies those, on a navigation problem whose obstacle layouts come from a published dataset rather than from a hand-drawn figure, and it reports a negative control — a floor without a localised hazard, on which the celebrated distinction vanishes entirely.

Chapter 3

Theoretical Background

This chapter develops the mathematics the implementation follows term by term. The verification suite of Chapter 6 checks these equations numerically.

3.1 Markov decision processes

A finite MDP is a tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ where $\mathcal{S}$ is a finite set of states, $\mathcal{A}$ a finite set of actions,

$$P(s' \mid s, a) = \Pr\{S_{t+1} = s' \mid S_t = s, A_t = a\} \tag{3.1}$$

the transition kernel, $R(s, a)$ the expected immediate reward, and $\gamma \in [0, 1)$ the discount factor. The defining assumption is the Markov property: the distribution of the next state depends on the present state and action alone.

For the warehouse of Chapter 4, $\mathcal{S}$ is the set of floor cells, $|\mathcal{S}| = 400$, and $\mathcal{A} = \{\text{up}, \text{down}, \text{left}, \text{right}\}$. The robot's cell is a sufficient statistic for its future, so the Markov property holds exactly rather than approximately — an advantage of this problem as an experimental vehicle.

3.1.1 Returns and the discount factor

The agent maximises the expected discounted return

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}. \tag{3.2}$$

Discounting serves two purposes: it keeps the sum finite in continuing tasks, and it expresses a preference for sooner rewards. With $\gamma = 0.99$ a reward n steps away retains 0.99^n of its value, so over the route lengths considered here (of order 20–40 steps) the goal reward remains clearly visible from the start state, which is what makes the task learnable at all.

3.2 Value functions and the Bellman equations

The action-value function of a policy π is

$$Q^{\pi}(s, a) = \mathrm{E}_{\pi}[G_t \mid S_t = s, A_t = a]\,, \tag{3.3}$$

and satisfies the Bellman expectation equation

$$Q^{\pi}(s,a) = \sum_{s'} P(s' \mid s,a)\Big[R(s,a) + \gamma \sum_{a'} \pi(a' \mid s')\, Q^{\pi}(s',a')\Big]. \tag{3.4}$$

The optimal action-value function satisfies the Bellman optimality equation

$$Q^{*}(s,a) = \sum_{s'} P(s' \mid s,a)\Big[R(s,a) + \gamma \max_{a'} Q^{*}(s',a')\Big]. \tag{3.5}$$

The entire distinction studied in this report is the difference between Equations 3.4 and 3.5. The expectation equation averages over the actions the policy will actually take; the optimality equation takes the maximum. SARSA is a sample-based solver for the first; Q-learning is a sample-based solver for the second. Everything that follows is a consequence of that one structural difference.

3.3 Temporal-difference learning

TD(0) estimates Q^{π} from experience without a model, by moving each estimate toward a bootstrapped target:

$$Q(S_t,A_t) \leftarrow Q(S_t,A_t) + \alpha\big[\underbrace{R_{t+1} + \gamma\, Q(S_{t+1},A_{t+1})}_{\text{TD target}} - Q(S_t,A_t)\big]. \tag{3.6}$$

The bracketed quantity is the *TD error* δ_t. Learning requires no knowledge of P or R: the environment supplies samples, and the update needs only the transition just experienced.

3.4 The two control algorithms

3.4.1 SARSA — on-policy control

SARSA applies TD(0) to the behaviour policy directly:

$$Q(S_t,A_t) \leftarrow Q(S_t,A_t) + \alpha\Big[R_{t+1} + \gamma\, Q(S_{t+1},A_{t+1}) - Q(S_t,A_t)\Big], \tag{3.7}$$

where A_{t+1} is drawn from the same ε-greedy policy the agent is following and is the action it will actually execute. The update is a sample of the right-hand side of Equation 3.4, so SARSA converges to Q^{π} for the ε-greedy policy π — not to Q^{*}, unless $\varepsilon \to 0$.

3.4.2 Q-learning — off-policy control

Q-learning replaces the sampled next action with a maximisation:

$$Q(S_t,A_t) \leftarrow Q(S_t,A_t) + \alpha\Big[R_{t+1} + \gamma \max_{a} Q(S_{t+1},a) - Q(S_t,A_t)\Big]. \tag{3.8}$$

This is a sample of Equation 3.5. The target no longer refers to the behaviour policy at all, so Q converges to Q^{*} under any behaviour policy that visits every state–action pair infinitely often.

3.4.3 The consequence, stated precisely

Consider a state s on the main aisle, with the pedestrian walkway directly above it. Let a_{safe} be the action along the aisle and a_{bad} the action into the walkway, with $R(s, a_{\text{bad}}) = -100$.

Under **Q-learning**, the value of s is backed up through $\max_a Q(s, a)$, which selects a_{safe}. The -100 never enters $V(s)$, because an optimal agent would never take that action. The aisle therefore looks exactly as good as any other corridor of the same length.

Under **SARSA** with an ε-greedy policy, the backup is through $Q(s, A')$ where A' is the action actually chosen, and

$$\mathrm{E}[Q(s, A')] = (1 - \varepsilon) \max_a Q(s, a) + \frac{\varepsilon}{|\mathcal{A}|} \sum_a Q(s, a), \tag{3.9}$$

which includes a term of weight $\varepsilon/|\mathcal{A}|$ on $Q(s, a_{\text{bad}})$. The -100 therefore propagates into $V(s)$, scaled by the probability of exploring into it. States beside the walkway are correspondingly devalued, and the safe lane becomes preferable.

This is the whole mechanism. It requires a hazard adjacent to the optimal route, a non-zero ε, and nothing else — in particular, no stochasticity in the environment.

3.5 The algorithms in full

For completeness, both algorithms are stated in the form implemented in Chapter 5. The lines marked $\star$ are the only point at which they differ.

Algorithm 1: Q-learning (off-policy TD control)

Initialise $Q(s, a) = 0$ for all $s \in \mathcal{S}, a \in \mathcal{A}$
for each episode **do**
    $s \leftarrow$ initial state; anneal ε
    repeat
        choose a from s using the ε-greedy policy derived from Q
        take action a, observe r and s'
        $\star$ $Q(s, a) \leftarrow Q(s, a) + \alpha\,[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$
        $s \leftarrow s'$
    until s is terminal
end for

Algorithm 2: SARSA (on-policy TD control)

Initialise $Q(s,a) = 0$ for all $s \in \mathcal{S}, a \in \mathcal{A}$
for each episode **do**
    $s \leftarrow$ initial state; anneal ε
    choose a from s using the ε-greedy policy derived from Q
    repeat
        take action a, observe r and s'
        choose a' from s' using the ε-greedy policy derived from Q
        $\star$ $Q(s,a) \leftarrow Q(s,a) + \alpha\,[r + \gamma\, Q(s',a') - Q(s,a)]$
        $s \leftarrow s'$; $a \leftarrow a'$
    until s is terminal
end for

Two structural differences follow from that single line. SARSA must select a' *before* the update, because the update consumes it; and it must then reuse that same a' on the next iteration, because otherwise the action it evaluated is not the action it took and the algorithm is no longer on-policy. Q-learning has no such constraint: its action for the next step is chosen freshly at the top of the loop, independently of the maximisation in its target.

3.6 Exploration and the epsilon schedule

An ε-greedy policy takes the greedy action with probability $1 - \varepsilon$ and a uniformly random action otherwise:

$$\pi(a \mid s) = \begin{cases} 1 - \varepsilon + \varepsilon/|\mathcal{A}| & \text{if } a = \arg\max_{a'} Q(s,a') \\ \varepsilon/|\mathcal{A}| & \text{otherwise.} \end{cases} \tag{3.10}$$

Both algorithms require sufficient exploration for their convergence guarantees. The schedule used here decays ε linearly from 1.0 to 0.05 over the first 60% of training and holds it thereafter. The residual $\varepsilon = 0.05$ is deliberate: were ε driven to zero, SARSA would converge to the same policy as Q-learning and the effect under study would disappear by construction.

3.7 Convergence conditions

Both algorithms converge in the tabular case under the Robbins–Monro step-size conditions

$$\sum_{t=1}^{\infty} \alpha_t = \infty, \qquad \sum_{t=1}^{\infty} \alpha_t^2 < \infty, \tag{3.11}$$

together with infinite visitation of every state–action pair [14, 5, 8].

This study uses a constant $\alpha = 0.10$, which violates the second condition. The choice is deliberate and standard in practice: a constant step size does not converge to a point but to a bounded region around it, tracking non-stationarity instead. Since both algorithms use the same α, the comparison is unaffected; and the asymptotic question is not what this study measures.

3.8 Complexity

Per timestep both algorithms perform $O(|\mathcal{A}|)$ work — a maximisation for Q-learning, an action sample for SARSA — and both store $O(|\mathcal{S}||\mathcal{A}|)$ values. For the warehouse here that is $400 \times 4 = 1600$ entries. The two are therefore indistinguishable in computational cost, which usefully removes one possible confound: any difference observed in the experiments is attributable to what is learned, not to how much computation was spent learning it.

Chapter 4

Dataset and Environment

4.1 What a dataset can and cannot supply

Supervised learning consumes rows. Reinforcement learning does not. Q-learning and SARSA learn by acting and observing the reward that follows, which means something must be able to answer the question *"what would have happened had the robot turned left instead?"*

A table of recorded observations cannot answer that. It records what did happen, not what would have. The counterfactual must come from a model of the dynamics — an environment. This is not a shortcoming of the particular dataset used here; it is a structural property of the problem class, and it determines what this study can honestly claim.

Table 4.1: What comes from the dataset and what is assumed. The distinction is maintained throughout the report.

Element	**Source**	**Note**
Racking layouts	Dataset	5,045 distinct 5×5 occupancy grids
Start and goal validity	Dataset	never placed on an obstacle in any row
Action encoding	Dataset	recovered empirically (Section 4.3)
Heuristic baseline	Dataset	from the logged `selected_action`
Floor composition	Assumed	4×4 tiling of real patches
Cross-aisle and walkway	Assumed	standard warehouse features
Reward constants	Assumed	declared, and swept in Chapter 7
Transition dynamics	Assumed	deterministic 4-connected movement

4.2 The dataset

The source is `zoya77/robot-path-planning-navigation-dataset` on Kaggle: 7,980 rows and 48 columns. Each row describes a warehouse snapshot — a 5×5 occupancy grid flattened across 25 columns, a start cell, a goal cell, the robot's current cell, an action index, and a number of derived quantities.

4.3 Audit

Several columns look informative and are not. Using them would silently corrupt the study, so each was tested before use.

4.3.1 The episode structure is not real

The file carries `episode_id` and `time_step` columns suggesting recorded trajectories. They do not describe trajectories:

- 5,727 of 7,980 rows duplicate the `(episode_id, time_step)` key.
- The obstacle grid changes *within* a single episode.
- The start cell changes within a single episode.

The rows are independently sampled snapshots. Both columns are excluded.

4.3.2 The supplied reward signal is noise

Table 4.2: Correlation of the supplied numeric columns with the true shortest-path distance, computed by breadth-first search.

Column	Correlation	Verdict
`reward`	−0.0298	no relationship with the task
`path_length_so_far`	+0.0105	no relationship with the task
`converged_flag`	—	constant 1, zero information
`joint_angle_1/2`	—	robot-arm angles, irrelevant to grid navigation

The `reward` column spans 0.90 to 1.00 over eleven values and is uncorrelated with anything. It is excluded, and the reward function is specified explicitly in Section 4.5 instead. There are likewise no reference shortest paths in the file, so ground truth is computed by breadth-first search, which is exact on a unit-cost grid and is itself verified in Chapter 6.

4.3.3 What is sound

Table 4.3: Properties of the usable portion of the dataset.

Property	Value
Distinct 5×5 layouts	5,045
Mean obstacles per grid	4.47 of 25
Start or goal placed on an obstacle	0 rows
Layouts with a reachable goal	96.1%
Median start-to-goal distance	3 steps

4.3.4 Recovering the action encoding

The `selected_action` column holds integers 0–3 with no documented meaning. The encoding was recovered by measuring the mean offset to the goal under each action: action 0 has mean row offset -1.70, action 1 has $+1.79$, action 2 has mean column offset -1.71, and action 3 has $+1.83$. The encoding is therefore up, down, left, right, and the notebook asserts this rather than assuming it.

This also reveals that the logged controller moves greedily toward the goal, which makes it usable as an external baseline in Chapter 7.

4.4 Constructing the floor

4.4.1 Why the raw grids are insufficient

The supplied layouts are 5×5 with a median start-to-goal distance of three steps. The distinction under study is a *choice* between a short risky route and a longer safe one, and at three steps there is no choice: the shortest path is the only path.

This was confirmed rather than assumed. On a floor built from scattered racking alone, both algorithms converge on the identical 39-step optimum. Scattered obstacles make the whole floor uniformly risky, so no safe-versus-risky decision exists anywhere in the state space. That negative control is reported in Chapter 8, because it establishes the condition under which the celebrated difference between these algorithms simply does not arise.

4.4.2 The floor used

Sixteen real 5×5 patches are tiled into a 20×20 floor of 400 cells, of which 67 (16.8%) are racking. Three rows are then cleared through the middle, modelling a main cross-aisle:

- **Row 10 — pedestrian walkway.** People. Entering it is a safety incident.
- **Row 11 — main aisle.** The direct route, running immediately alongside the walkway.
- **Row 12 — safe lane.** A parallel route one row further from people.

The robot starts at $(11, 0)$ and must reach $(11, 19)$. The optimal legal route is **19 steps** straight along the main aisle, confirmed by breadth-first search.

This is the structure of Cliff Walking [11] expressed in warehouse terms. It is stated as a design choice, not presented as something the dataset provided; without a hazard adjacent to the optimal route there is no effect to measure.

4.5 The Markov decision process

States. The robot's cell, indexed $s = \text{row} \times W + \text{col}$, giving $|\mathcal{S}| = 400$. The cell is a sufficient statistic for the future, so the Markov property holds exactly.

Actions. The four moves recovered from the data.

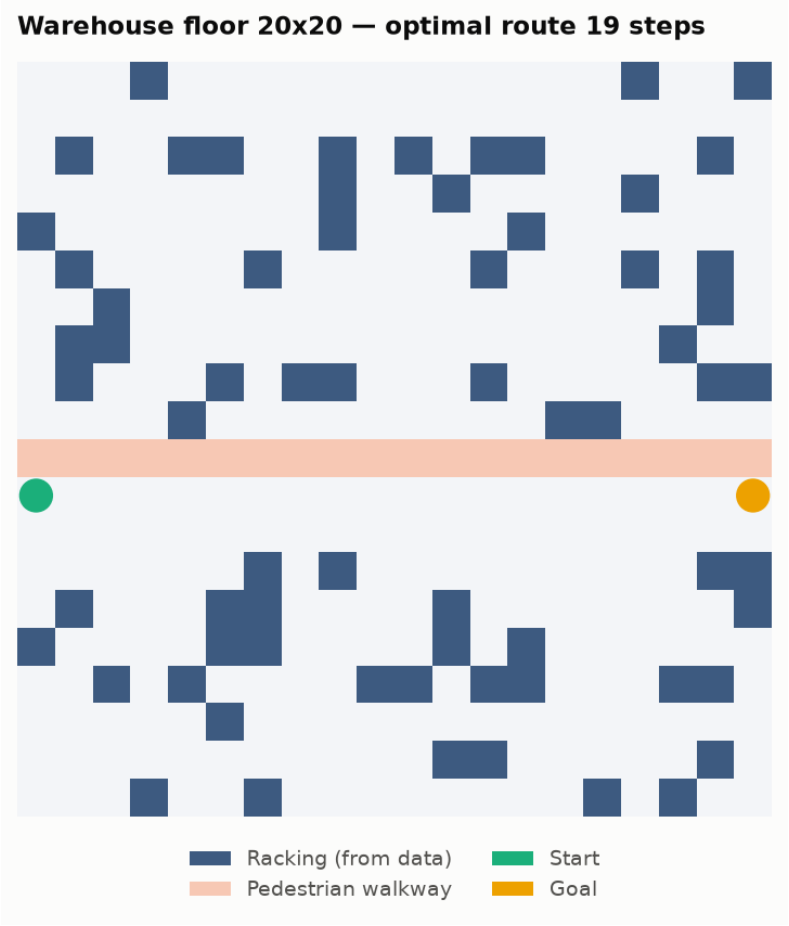


Figure 4.1: The warehouse floor. Racking in dark blue comes from sixteen real 5×5 dataset patches; the pale band is the pedestrian walkway. The robot travels from the green charging station to the amber delivery point.

Transitions. Deterministic. A slip probability is implemented and defaults to zero, since the effect under study is driven by ε-greedy exploration rather than by environmental stochasticity.

Rewards. These are the modelling assumptions:

Separating "bump" from "incident" is essential and was learned the hard way. An earlier design ended the episode on any collision. Because 17% of the floor is racking, a random walk struck a shelf within a few steps, every episode terminated almost immediately, and neither algorithm ever reached the goal to discover the reward — measured training success was 0.01. Distinguishing a harmless bump from a genuine hazard is what makes the problem learnable while preserving the risk structure the study requires.

Table 4.4: The reward structure. Every value here is a declared assumption; Chapter 7 varies the one that drives the result.

Event	**Reward**	**Effect**	**Justification**
Ordinary step	-1	moves	time costs money
Nudging the racking	-5	stays put	a bump, not an incident
Entering the walkway	-100	returned to start	a safety incident
Reaching the goal	$+100$	episode ends	delivery completed

Chapter 5

Implementation

5.1 The from-scratch constraint

Both algorithms and the environment are written from first principles in NumPy. No reinforcement learning library is used: there is no `gymnasium`, no `stable-baselines`, and no pre-built agent. NumPy supplies array arithmetic and random number generation; everything specific to the MDP, to temporal-difference control, and to the two update rules is written out.

5.2 The environment

`WarehouseEnv` is a plain class with `reset()` and `step()`.

```
def step(self, action):
    self.t += 1
    dr, dc = MOVES[action]
    nr, nc = self.pos[0] + dr, self.pos[1] + dc
    info = {}

    if not (0 <= nr < self.H and 0 <= nc < self.W) or self.grid[nr, nc] == 1:
        reward, done = self.r_bump, False        # nudge: no movement
        info["bump"] = True
    elif self.walkway[nr, nc]:
        self.pos = self.start                    # incident: back to charger
        reward, done = self.r_incident, False
        info["incident"] = True
    else:
        self.pos = (nr, nc)
        if self.pos == self.goal:
            reward, done, info["goal"] = self.r_goal, True, True
        else:
            reward, done = self.r_step, False
    if self.t >= self.max_steps:
        done, info["timeout"] = True, True
    return self.encode(self.pos), reward, done, info
```

The three outcomes are mutually exclusive and ordered deliberately: a cell that is both racking and walkway cannot occur by construction, and the boundary check precedes the grid lookup so that an out-of-bounds index is never formed.

State encoding is $s = r \cdot W + c$ with the inverse $(s \div W,\ s \bmod W)$. Bijectivity over all 400 cells is asserted in Chapter 6.

5.3 The shared agent

The two algorithms share a base class holding everything they have in common: the action-value table, ε-greedy action selection with random tie-breaking, and the ε schedule.

```
class TDControlAgent:
    def act(self, s):
        if self.rng.random() < self.eps:
            return int(self.rng.integers(self.n_actions))
        return self.greedy(s)

    def greedy(self, s):
        q = self.Q[s]
        return int(self.rng.choice(np.flatnonzero(q == q.max())))   # random tie-break

    def update(self, s, a, r, s_next, a_next, done):
        tgt = self._target(r, s_next, a_next, done)
        self.Q[s, a] += self.alpha * (tgt - self.Q[s, a])
```

Random tie-breaking matters more than it appears. With Q initialised to zeros every action ties initially, and breaking ties by `argmax` would bias the agent toward action 0 — "up", directly into the walkway — producing an artefact that looks like a finding.

5.4 The one-line difference

The subclasses override a single method. This is the entire distinction between the two algorithms, and the code is arranged so that nothing else can differ:

```
class QLearningAgent(TDControlAgent):
    # Off-policy: bootstraps from the best available next action.
    name = "Q-learning"
    def _target(self, r, s_next, a_next, done):
        return r if done else r + self.gamma * self.Q[s_next].max()


class SarsaAgent(TDControlAgent):
    # On-policy: bootstraps from the action actually selected for the next step.
    name = "SARSA"
    def _target(self, r, s_next, a_next, done):
        return r if done else r + self.gamma * self.Q[s_next, a_next]
```

These implement Equations 3.8 and 3.7 respectively. Note the terminal handling: when `done` is true the target is r alone, with no bootstrap term, since a terminal state has value zero by definition. Omitting that condition is a common error which inflates all values by a spurious $\gamma V(s_{\text{terminal}})$ term.

5.5 The episode loop

```
def run_episode(env, agent, learn=True, greedy=False):
```

```
s = env.reset()
a = agent.greedy(s) if greedy else agent.act(s)
while True:
    s_next, r, done, info = env.step(a)
    a_next = (agent.greedy(s_next) if greedy
              else agent.act(s_next)) if not done else 0
    if learn:
        agent.update(s, a, r, s_next, a_next, done)
    if done: break
    s, a = s_next, a_next
```

The critical detail is that `a_next` is chosen before the update and then actually executed on the following iteration, via `s, a = s_next, a_next`. SARSA is on-policy precisely because the action it bootstraps from is the action it takes. Sampling a fresh action after the update — an easy and common slip — silently converts SARSA into an inconsistent hybrid that still trains and still produces a plausible learning curve. The verification suite is designed to catch exactly this.

5.6 Experimental harness

A single `train()` function serves both algorithms, so the training budget is identical by construction rather than by careful duplication.

Table 5.1: Hyperparameters. Every value is shared between the two algorithms.

Parameter	**Value**	**Note**
Learning rate α	0.10	constant
Discount γ	0.99	
ε schedule	$1.0 \rightarrow 0.05$	linear over the first 60% of training
Episodes per run	3,000	
Episode cap	600 steps	
Seeds	30	paired across algorithms

The residual $\varepsilon = 0.05$ is deliberate. Driving ε to zero would make SARSA converge to the same policy as Q-learning, eliminating the effect under study by construction.

Each seed trains both a Q-learning and a SARSA agent on the same floor with the same budget, the same hyperparameters and the same schedule. Evaluation is then performed separately with $\varepsilon = 0$, which distinguishes the policy the agent *learned* from the behaviour it exhibited while learning — the distinction on which the entire study turns.

Chapter 6

Verification

6.1 Why this chapter precedes the results

A reinforcement learning defect rarely announces itself. A mis-signed reward, an off-by-one in the state encoding, or SARSA bootstrapping from a freshly sampled action rather than the one it will execute — each of these still produces a learning curve that rises, an agent that improves, and a comparison that looks entirely respectable. The agent learns *something*; it is simply not what the experiment claims to be measuring.

Thirteen checks therefore run before any experiment, and the suite asserts on failure so that results cannot be produced from an unverified implementation.

6.2 Environment checks

Table 6.1: Checks on the environment.

Check	**Rules out**	**Result**
State encoding bijective	index collisions	400/400
Bump leaves position unchanged	movement bookkeeping	pass
Bump: penalty, no termination	wrong episode semantics	-5, continues
Incident returns robot to start	hazard transition	pass
Incident: penalty, no termination	wrong episode semantics	-100, continues
Goal: reward and termination	double-counted terminal	$+100$, ends
Return accounting matches env	drift between env and agent	exact

6.3 Ground-truth checks

Breadth-first search provides the optimality reference, so it is itself verified: against a hand-checked 5×5 grid whose answer (8 steps) was counted by hand, and against a deliberately disconnected grid where it must report unreachability rather than a spurious distance. A third check confirms the optimal route respects the walkway exclusion.

6.4 The decisive check

At $\varepsilon = 0$, the two algorithms must be bit-identical.

Under a greedy policy the action selected at s' *is* the maximising action, so $\max_{a'} Q(s', a')$ and $Q(s', a')$ are the same quantity and Equations 3.8 and 3.7 coincide exactly. Two agents trained from the same seed with ε fixed at zero must therefore produce identical action-value tables.

Measured: $\max |Q_{\text{Q-learning}} - Q_{\text{SARSA}}| = 0.00 \times 10^{0}$ — identical to the last bit.

This is the strongest statement available about the implementation. It establishes that the two classes differ *only* in how exploration is handled, which is the precise claim the entire study rests on. Had SARSA been sampling a fresh action instead of reusing the one it executes, the tables would have diverged here even with $\varepsilon = 0$.

6.5 Learning checks

Both agents are required to recover the known optimal policy on a 3×3 open grid, where the optimum is four steps. Both do. A forward pass can be correct while gradients, or in this case value propagation, fail to reach the parameters; this check closes that gap.

6.6 Results

Table 6.2: All thirteen checks, executed before every experiment.

#	**Check**	**Result**
1	State encoding bijective	pass
2	Bump: robot does not move	pass
3	Bump: penalty, episode continues	pass
4	Incident: returned to start	pass
5	Incident: penalty, episode continues	pass
6	Goal: reward and termination	pass
7	BFS matches hand-checked grid	pass
8	BFS reports unreachable as -1	pass
9	Optimal route avoids the walkway	pass
10	$\varepsilon = 0$: the two agents are identical	0.00×10^{0}
11	Q-learning optimal on a 3×3 grid	4 of 4 steps
12	SARSA optimal on a 3×3 grid	4 of 4 steps
13	Return accounting matches the environment	exact

6.7 What this does not establish

The suite establishes that the environment and the agents compute what they are intended to compute. It does not establish that the *experimental design* is sound — a poor choice of reward constants, or a floor without a hazard, would pass every check above and still produce a meaningless comparison.

That is why Chapter 7 includes an external baseline, a generality check and a sensitivity sweep, and why Chapter 8 reports a negative control.

Chapter 7

Results

All results are from thirty paired seeds of 3,000 episodes each on the floor of Chapter 4, whose optimal legal route is **19 steps**.

7.1 Headline comparison

Table 7.1: Means over thirty seeds. "Online return" is the mean episode return during training, with exploration active; "deployed" figures come from the greedy policy evaluated at $\varepsilon = 0$ after training.

Measure	Q-learning	SARSA	Optimal
Online return (training)	−838.33	**−297.32**	—
Online return, final 20%	57.57	**63.18**	—
Walkway incidents in training	21,626	**5,068**	0
Deployed route length (steps)	**19.40**	33.34	19
Deployed success rate	1.00	0.97	1.00
Mean distance from the walkway	1.00	**5.21**	1.00

The two algorithms separate exactly as the theory of Chapter 3 predicts, and they separate in *opposite directions* on the two questions:

- **Q-learning produces the better deployed policy.** At 19.40 steps against an optimum of 19, it recovers the shortest legal route essentially perfectly.
- **SARSA is far cheaper to train.** It incurs 5,068 walkway incidents against Q-learning's 21,626 — a factor of 4.3 — and its online return is better by a wide margin.

7.2 Statistical significance

Because both algorithms ran on the same thirty seeds, every metric yields thirty paired differences.

Four of the five differences are significant at $\alpha = 0.05$, and the effects are large rather than marginal. The confidence interval on deployed route length is $[14, 14]$ — degenerate, because every one of the thirty seeds produced the same 14-step difference, which is itself an indication of how consistently the two algorithms converge to their respective routes.

Table 7.2: Paired comparison, SARSA minus Q-learning, over 30 seeds. Confidence intervals are bootstrap percentile intervals on the median difference.

Metric	Median diff.	95% CI	Wilcoxon p	Verdict
Online return	+545.15	[493.60, 586.44]	1.9×10^{-9}	significant
Online return, late	+8.81	[6.81, 10.19]	3.5×10^{-3}	significant
Training incidents	−16,608	[−17891, −15322]	1.9×10^{-9}	significant
Deployed route steps	+14.00	[14.00, 14.00]	6.7×10^{-7}	significant
Deployed success	0.00	[0.00, 0.00]	0.317	not significant

The only non-significant difference is deployed success rate: Q-learning reached the goal in all thirty runs and SARSA in 29 of 30, a difference the test cannot distinguish from chance.

7.3 The learned routes

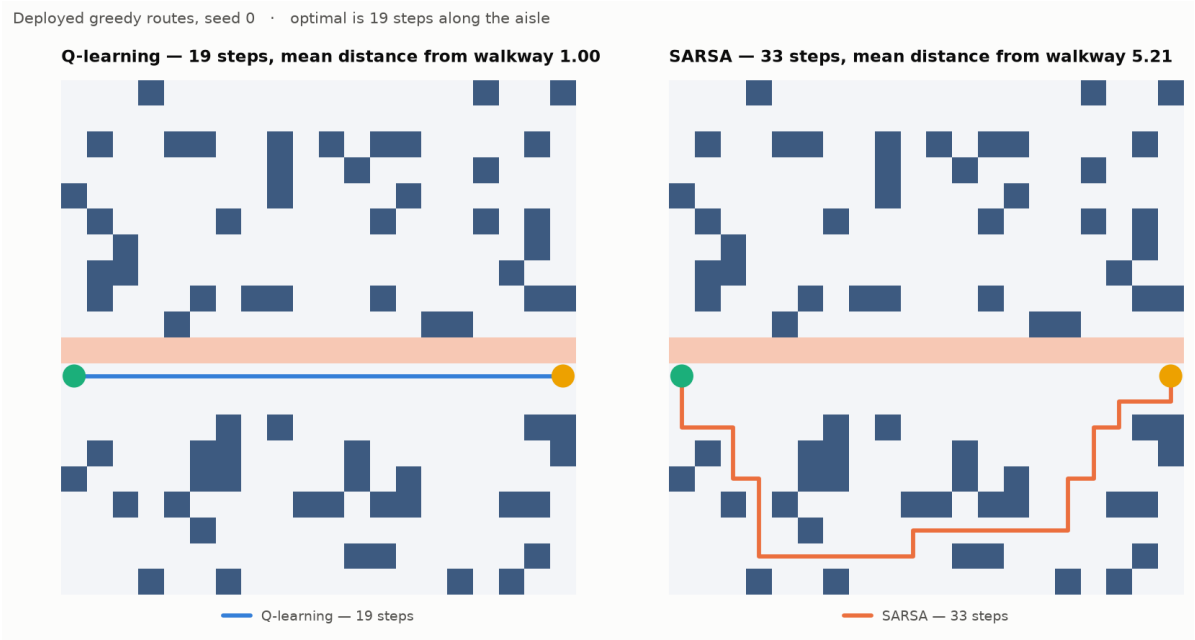


Figure 7.1: The deployed greedy routes, seed 0. Q-learning (left) runs dead straight along the main aisle, immediately beneath the pedestrian walkway, in the optimal 19 steps. SARSA (right) descends into the racking and takes 33 steps, averaging 5.21 cells from the walkway rather than 1.00.

Figure 7.1 is the clearest statement of the result in the report. The two algorithms were given identical data, identical hyperparameters and an identical experience budget, and differed only in which policy's value they estimated. One produced a straight line along the hazard; the other produced a detour that never approaches it.

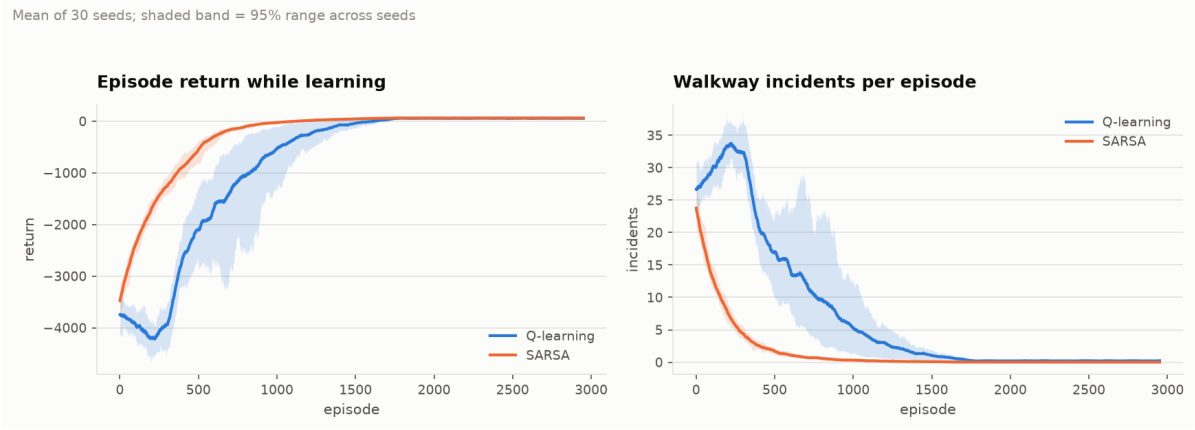


Figure 7.2: Episode return and walkway incidents during training, averaged over thirty seeds with 95% bands. SARSA's advantage in online return is visible throughout, and is driven by the incident rate in the right-hand panel.

Table 7.3: Deployed policies against external references.

Policy	**Steps**	**Success**	**Incidents**	**Return**
Optimal (BFS)	19.00	1.00	0	82.00
Heuristic (from the data)	19.00	1.00	0	82.00
Q-learning	19.40	1.00	0	81.60
SARSA	33.34	0.97	0	−31.93

7.4 External baselines

Two observations deserve emphasis, and the second is unflattering.

First, **neither deployed policy causes any incidents**. Both algorithms learned to keep out of the walkway; the difference between them is how much margin they leave and what that margin cost to acquire.

Second, **the heuristic controller recovered from the dataset matches the optimum exactly**, at 19 steps with zero incidents — equalling Q-learning and beating SARSA. This should be stated plainly rather than buried: on this particular floor, a controller that simply moves toward the goal is optimal, because the main aisle happens to run straight from start to goal. The heuristic has no mechanism for handling racking that blocks its preferred direction and would fail on a floor requiring a detour, as the generality check below indirectly shows. But on this instance it wins, and the learned agents have not demonstrated superiority over it.

7.5 Generality across floors

The pattern holds on every floor: Q-learning closer to optimal, SARSA with roughly a third of the incidents. A paired Wilcoxon test on training incidents across the twelve floors gives $p = 4.9 \times 10^{-4}$. The finding is not an artefact of one sampled racking arrangement.

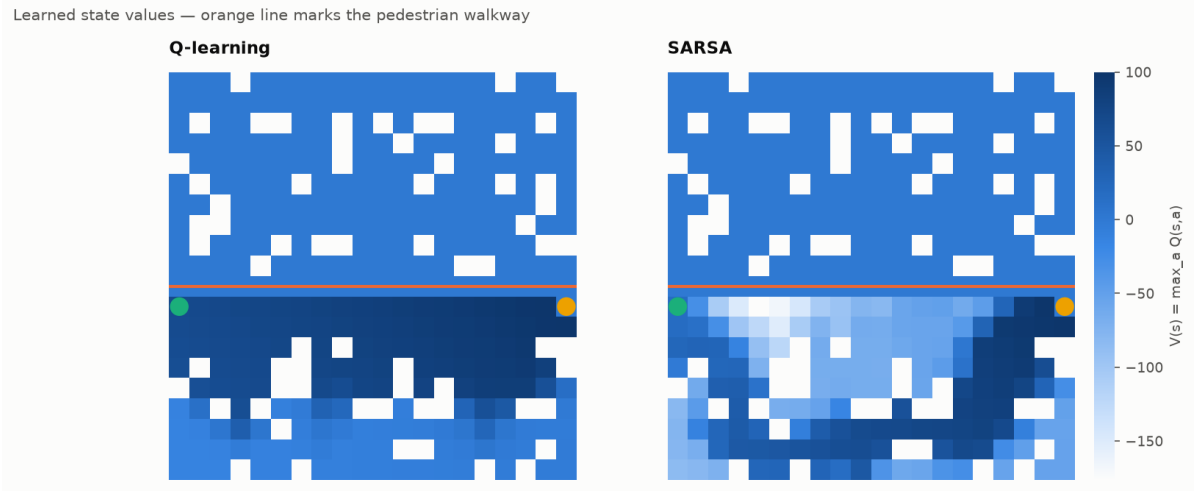


Figure 7.3: Learned state values $V(s) = \max_a Q(s, a)$, with the walkway marked in orange. Q-learning assigns high value to the aisle immediately beneath the walkway; SARSA's values are depressed in that region, which is the devaluation derived in Chapter 3 appearing directly in the learned table.

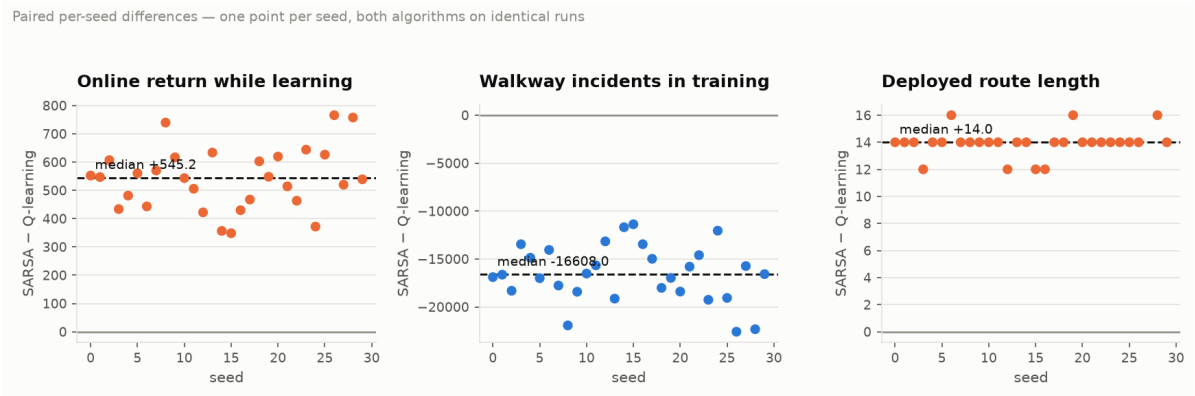


Figure 7.4: Per-seed paired differences. Each point is one seed; both algorithms were trained under identical conditions for that seed.

7.6 Sensitivity to the hazard penalty

The hazard penalty is the central modelling assumption, so the experiment was repeated across a sweep from 0 to −200.

This table contains the single most informative result in the report, and it is not the headline one.

Q-learning's behaviour is completely invariant to the hazard penalty. Its incident count is identical — 6,522 — at every penalty from −10 to −200, and its distance from the walkway is 1.74 throughout. A twenty-fold increase in the cost of an incident changes its behaviour by nothing at all.

This is exactly what Chapter 3 predicts. Q-learning backs up through $\max_a Q(s, a)$, which selects the safe action, so the penalty attaches to $Q(s, a_{\text{bad}})$ but never propagates into $V(s)$. The states beside the walkway are therefore never devalued however large the penalty becomes. The mechanism is not inferred from the outcome here — it is directly visible in the measurements.

SARSA responds monotonically. Its clearance rises from 1.91 at zero penalty to 5.39 at −200, and its incidents fall correspondingly. The on-policy backup carries an $\varepsilon/|\mathcal{A}|$-weighted share of the penalty

Table 7.4: Twelve independently composed floors, each with its own racking sampled from the dataset. "Excess steps" is the deployed route length minus the BFS optimum for that floor.

Algorithm	Success	Excess steps	Incidents	Online return
Q-learning	1.00	**2.17**	5,927	−700.64
SARSA	1.00	11.33	**1,792**	**−293.91**

Table 7.5: Sensitivity to the hazard penalty, eight seeds per cell. Q-learning's behaviour does not change at all; SARSA's clearance rises monotonically with the penalty.

Penalty	Training incidents		Distance from walkway	
	Q-learning	SARSA	Q-learning	SARSA
0	401,268	4,157	—	1.91
−10	6,522	2,817	1.74	2.27
−25	6,522	2,307	1.74	4.84
−50	6,522	2,060	1.74	5.15
−100	6,522	1,947	1.74	5.25
−200	6,522	1,844	1.74	5.39

into the state value, so a larger penalty produces a wider berth.

At zero penalty the effect disappears, and Q-learning degenerates. With no cost, entering the walkway is a free teleport back to the start, and Q-learning's greedy policy cycles through it indefinitely — 401,268 incidents, and it fails to reach the goal at all. This is the degenerate end of the sweep rather than a tuning failure, and it is reported because it demonstrates that the hazard penalty is not a free parameter that can be set to taste.

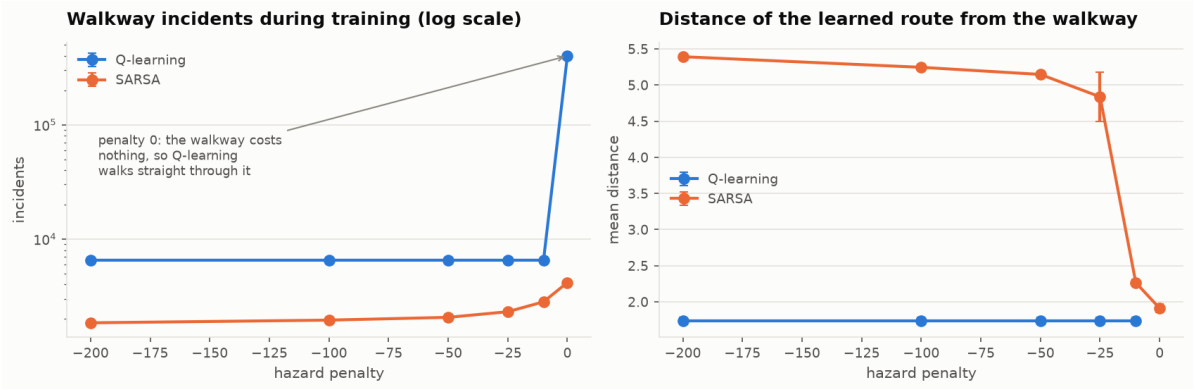


Figure 7.5: Sensitivity to the hazard penalty. Incidents are plotted on a log scale because the zero-penalty case is two orders of magnitude above the rest.

Chapter 8

Discussion

8.1 Answering the research question

> *How do Q-learning and SARSA compare when a warehouse robot must learn the shortest collision-free route to a delivery point?*

8.1.1 They are not better or worse; they answer different questions

Q-learning produced a deployed route of 19.40 steps against an optimum of 19. SARSA produced 33.34. On deployed quality, Q-learning wins decisively, and the difference is significant at $p = 6.7 \times 10^{-7}$.

Q-learning accumulated 21,626 walkway incidents while learning; SARSA accumulated 5,068. On the cost of learning, SARSA wins by a factor of 4.3, significant at $p = 1.9 \times 10^{-9}$.

Both statements are true simultaneously, and neither is a ranking of the algorithms. They are answers to two different questions that are frequently conflated: *how good is the policy you end up with* and *what did it cost to get there.*

8.1.2 The mechanism, confirmed directly

Chapter 3 derived the mechanism: Q-learning's target maximises over next actions, so a hazard reachable only by a non-greedy action never enters the state value, while SARSA's target carries an $\varepsilon/|\mathcal{A}|$-weighted share of it.

The sensitivity sweep confirms this without relying on inference. Q-learning's incident count is numerically identical — 6,522 — at hazard penalties of -10, -25, -50, -100 and -200, and its route keeps a constant distance of 1.74 from the walkway. Scaling the cost of an incident by a factor of twenty changes its behaviour by nothing whatsoever, because the penalty is structurally invisible to its state values. SARSA's clearance, over the same sweep, rises monotonically from 1.91 to 5.39.

It is rare for a derivation and a measurement to line up this cleanly, and it is worth stating that the agreement is what gives confidence that the implementation is measuring the intended quantity rather than an artefact.

8.1.3 The effect is a response to risk, not a property of the algorithms

Chapter 4 reported a negative control that deserves more prominence than its placement there suggests. On a floor built from scattered racking with no localised hazard, both algorithms converged on the *identical* 39-step optimum. There was no difference to measure.

The same conclusion appears at the other end of the sensitivity sweep: at zero hazard penalty the clearance gap between the algorithms nearly closes.

The on-policy/off-policy distinction is therefore not a general characteristic that one algorithm is safe and the other is greedy. It is a response to a specific structural property of the task — a high-penalty region adjacent to the optimal path. Where that property is absent, so is the effect. Secondary accounts of Cliff Walking that present SARSA as "the safe algorithm" omit this condition, and the omission matters: on a task without an adjacent hazard, choosing SARSA buys nothing and costs route length.

8.1.4 Which to use, and when

The decision follows from where learning happens.

Learning in simulation, deploying the result. Q-learning. Training incidents are free because they occur in a simulator, and its asymptotic optimality is exactly what is wanted. This describes most industrial practice.

Learning on a live floor among people. SARSA, or an explicit safe-exploration method. Every incident is real, and a 4.3-fold reduction in incidents during training is worth a 14-step detour that can be re-optimised later.

Either, followed by ε decay to zero. Both converge to the optimal policy in the limit under GLIE conditions [8]. The residual $\varepsilon = 0.05$ used here was retained deliberately to keep the distinction visible.

8.2 An uncomfortable comparison

The heuristic controller recovered from the dataset — move toward the goal — achieved the optimal 19 steps with zero incidents, equalling Q-learning and beating SARSA.

This should not be waved away. On this floor, that controller is optimal, because the main aisle runs straight from the start to the goal and "move toward the goal" is therefore sufficient. The learned agents spent 3,000 episodes to match what a four-line rule achieves immediately.

The honest reading is that the warehouse floor used here is a good instrument for isolating the on-policy/off-policy distinction, and a poor advertisement for reinforcement learning as a solution to this particular routing problem. A greedy heuristic fails when the direct direction is blocked and a detour is required, which is precisely the situation the generality check across twelve floors begins to probe — but that check was not designed to compare against the heuristic, and so this report does not claim the learned policies are superior to it. Establishing that would require floors where the heuristic demonstrably fails, and is identified as future work.

8.3 Threats to validity

8.3.1 Internal validity

The reward constants are assumed. The dataset supplied no usable reward signal, so the step cost, bump penalty, hazard penalty and goal reward were chosen. The sensitivity sweep quantifies the dependence on the one that drives the result, and shows the conclusion holds across -10 to -200 while disappearing at 0.

A constant step size. $\alpha = 0.10$ violates the Robbins–Monro square-summability condition, so neither algorithm converges to a point. Both use the same value, so the comparison is unaffected, but asymptotic claims are not supported by this design.

Shared hyperparameters. α, γ and the ε schedule were fixed and shared rather than tuned per algorithm. This is the correct choice for a controlled comparison, but a schedule better suited to one

algorithm could shift the balance.

8.3.2 External validity

The hazard is a construction. The racking is real; the cross-aisle and the walkway are not. Without them there is no effect, so this is not a neutral design decision and is not presented as one. What the study establishes is the behaviour of the two algorithms *given* such a structure, together with evidence about what happens when it is absent.

One floor geometry. All floors share the same three-row cross-aisle arrangement; only the racking varies. A different hazard topology — scattered restricted zones, or a walkway crossing the route rather than running parallel to it — might behave differently.

Tabular only. 400 states are enumerable. Under function approximation, off-policy bootstrapping interacts with the deadly triad [11] in ways this study cannot speak to.

Deterministic dynamics. Slip probability was zero. Stochastic transitions would be expected to widen the gap, since SARSA's conservatism also hedges against randomness, but this was not measured.

8.3.3 Construct validity

"Incidents" are a proxy. Counting walkway entries treats every incident as equally costly, which a real safety analysis would not.

Online return mixes two things. It aggregates route efficiency and incident cost into one number. The report therefore also presents incident counts separately, since they are the quantity an operator would actually care about.

Chapter 9

Conclusion and Future Work

9.1 Summary

This report compared Q-learning and SARSA on a warehouse navigation task in which the shortest route runs alongside a pedestrian walkway. Both algorithms were implemented from first principles, sharing every component except the single expression forming their update target, and were verified against thirteen numerical checks before any experiment was run — including an equivalence test confirming that at $\varepsilon = 0$ they produce bit-identical action-value tables.

The comparison used thirty paired seeds with Wilcoxon signed-rank tests and bootstrap confidence intervals, addressing directly the single-seed limitation that was the principal weakness of the predecessor study in this repository.

9.2 Findings

Finding 1 — the two algorithms separate in opposite directions on two different questions. Q-learning's deployed route averaged 19.40 steps against a BFS optimum of 19; SARSA's averaged 33.34 ($p = 6.7 \times 10^{-7}$). Q-learning accumulated 21,626 walkway incidents during training against SARSA's 5,068, a factor of 4.3 ($p = 1.9 \times 10^{-9}$). Four of five metrics differed significantly, with large effects.

Finding 2 — the mechanism is directly observable, not merely inferred. Q-learning's behaviour is numerically invariant to the hazard penalty: identical incident counts and identical clearance at -10, -25, -50, -100 and -200. Because its target maximises over next actions, a penalty reachable only by a non-greedy action never enters the state value. SARSA's clearance rises monotonically over the same sweep, from 1.91 to 5.39.

Finding 3 — the effect requires a localised hazard and vanishes without one. On a floor of scattered racking with no walkway, both algorithms converged on the identical optimum. The distinction is a response to a property of the task, not a general ranking.

Finding 4 — a four-line heuristic matched the better of the two. The controller recovered from the dataset achieved the optimal 19 steps with zero incidents. On this floor, neither learned agent demonstrated superiority over it.

9.3 Future work

Floors where the heuristic fails. The most valuable extension, since it is the weakness identified in Chapter 8. Constructing layouts that require detours around blocking racking would establish whether

the learned policies offer anything the greedy rule does not.

Expected SARSA and Double Q-learning. Expected SARSA [12] replaces the sampled bootstrap with its expectation under the policy and should sit between the two algorithms studied here, with lower variance; Double Q-learning [4] would address the maximisation bias the max operator introduces.

Decaying ε to zero. Under GLIE conditions both algorithms converge to the optimal policy. Measuring the rate at which SARSA's route shortens as $\varepsilon \to 0$ would quantify how much of its conservatism is permanent and how much is a function of the exploration schedule.

Stochastic dynamics. Non-zero slip probability would test whether SARSA's advantage widens when the environment, and not only the policy, is a source of randomness.

Function approximation. Scaling beyond enumerable state spaces would bring the off-policy stability question into play, which the tabular setting excludes by construction.

9.4 Closing remarks

The most useful result here is not the headline comparison, which reproduces a textbook example. It is the sensitivity sweep, which shows Q-learning's behaviour to be entirely unmoved by a twenty-fold change in the cost of an incident — and the negative control, which shows that on a task without an adjacent hazard the distinction between these two celebrated algorithms does not exist at all.

Both point the same way. An algorithm's advantages are conditional on the task exercising the property that algorithm addresses, and the conditions matter as much as the result.

Bibliography

[1] R. Bellman, *Dynamic Programming*. Princeton University Press, 1957.

[2] P. Dayan, "The convergence of TD(λ) for general λ," *Machine Learning*, vol. 8, pp. 341–362, 1992.

[3] J. García and F. Fernández, "A comprehensive survey on safe reinforcement learning," *JMLR*, vol. 16, pp. 1437–1480, 2015.

[4] H. van Hasselt, "Double Q-learning," in *Proc. NeurIPS*, 2010.

[5] T. Jaakkola, M. I. Jordan, and S. P. Singh, "On the convergence of stochastic iterative dynamic programming algorithms," *Neural Computation*, vol. 6, no. 6, pp. 1185–1201, 1994.

[6] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Wiley, 1994.

[7] G. A. Rummery and M. Niranjan, "On-line Q-learning using connectionist systems," Technical Report CUED/F-INFENG/TR 166, Cambridge University Engineering Department, 1994.

[8] S. Singh, T. Jaakkola, M. L. Littman, and C. Szepesvári, "Convergence results for single-step on-policy reinforcement-learning algorithms," *Machine Learning*, vol. 38, pp. 287–308, 2000.

[9] R. S. Sutton, "Learning to predict by the methods of temporal differences," *Machine Learning*, vol. 3, pp. 9–44, 1988.

[10] R. S. Sutton, "Generalization in reinforcement learning: successful examples using sparse coarse coding," in *Proc. NeurIPS*, 1996.

[11] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[12] H. van Seijen, H. van Hasselt, S. Whiteson, and M. Wiering, "A theoretical and empirical analysis of Expected Sarsa," in *Proc. IEEE ADPRL*, 2009.

[13] C. J. C. H. Watkins, "Learning from delayed rewards," Ph.D. dissertation, University of Cambridge, 1989.

[14] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, pp. 279–292, 1992.

[15] C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.

[16] P. Virtanen et al., "SciPy 1.0: fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

Appendix A

Reproduction

Every figure and number in this report is produced by executing `rl_warehouse_qlearning_vs_sarsa.ipynb`. The repository is at `https://github.com/Ashwin-3cS/transformer-t1`.

A.1 Environment

```
uv venv .venv --python 3.14
uv pip install --python .venv/bin/python numpy pandas scipy matplotlib kaggle ipykernel jupyter
mkdir -p ~/.kaggle
echo KGAT_xxxxxxxx > ~/.kaggle/access_token
chmod 600 ~/.kaggle/access_token
```

A.2 Execution

```
MPLBACKEND=Agg .venv/bin/jupyter nbconvert --to notebook --execute --inplace \
    --ExecutePreprocessor.timeout=10800 rl_warehouse_qlearning_vs_sarsa.ipynb
```

Runtime is roughly four minutes on CPU: thirty paired seeds of 3,000 episodes, twelve floors for the generality check and six penalty settings for the sensitivity sweep. Setting `SMOKE_TEST = True` reduces this to about a minute.

Outputs are written to `artifacts/rl/`: `metrics.json`, `per_seed_results.csv` and the six figures reproduced in this report.

A.3 Determinism

All generators are seeded. The experiment was executed twice during preparation; every reported metric, including the full McNemar-style paired table, was identical between runs.

Appendix B

Source Listings

B.1 The two agents

```
class TDControlAgent:
    def __init__(self, n_states, n_actions, alpha=ALPHA, gamma=GAMMA,
                 eps_start=EPS_START, eps_end=EPS_END, seed=SEED):
        self.Q = np.zeros((n_states, n_actions), dtype=np.float64)
        self.n_actions = n_actions
        self.alpha, self.gamma = alpha, gamma
        self.eps_start, self.eps_end, self.eps = eps_start, eps_end, eps_start
        self.rng = np.random.default_rng(seed)

    def set_epsilon(self, frac_done, decay_frac=EPS_DECAY_FRAC):
        f = min(1.0, frac_done / decay_frac) if decay_frac > 0 else 1.0
        self.eps = self.eps_start + f * (self.eps_end - self.eps_start)

    def act(self, s):
        if self.rng.random() < self.eps:
            return int(self.rng.integers(self.n_actions))
        return self.greedy(s)

    def greedy(self, s):
        q = self.Q[s]
        return int(self.rng.choice(np.flatnonzero(q == q.max())))

    def update(self, s, a, r, s_next, a_next, done):
        self.Q[s, a] += self.alpha * (self._target(r, s_next, a_next, done) - self.Q[s, a])


class QLearningAgent(TDControlAgent):
    name = "Q-learning"
    def _target(self, r, s_next, a_next, done):
        return r if done else r + self.gamma * self.Q[s_next].max()


class SarsaAgent(TDControlAgent):
    name = "SARSA"
    def _target(self, r, s_next, a_next, done):
        return r if done else r + self.gamma * self.Q[s_next, a_next]
```

B.2 Floor construction

```
def build_warehouse(patch_pool, tile=TILE, rng=None):
    # Racking from the data; the cross-aisle and walkway are declared assumptions.
    grid = compose_floor(patch_pool, tile, rng)
    H_, W_ = grid.shape
    rw = H_ // 2
    grid[rw, :] = 0                                 # walkway   (people)
    grid[rw + 1, :] = 0                             # main aisle (the direct route)
    grid[rw + 2, :] = 0                             # safe lane  (the detour)
    walkway = np.zeros_like(grid, dtype=bool)
    walkway[rw, :] = True
    start, goal = (rw + 1, 0), (rw + 1, W_ - 1)
    return grid, walkway, start, goal
```

Appendix C

Verification Output

Produced on every execution, before training begins.

```
check                                              result  detail
-------------------------------------------------------------------------------
State encoding bijective                           PASS    400 codes for 400 cells
Bump: robot does not move                          PASS    stayed at (1, 1)
Bump: penalty, episode continues                   PASS    reward=-5.0, done=False
Incident: returned to start                        PASS    pos=(1,1), start=(1,1)
Incident: penalty, episode continues               PASS    reward=-100.0, done=False
Goal: reward and termination                       PASS    reward=100.0, done=True
BFS matches hand-checked grid                      PASS    BFS=8, expected 8
BFS reports unreachable as -1                      PASS
Optimal route avoids the walkway                   PASS    legal route 19 steps
eps=0: Q-learning and SARSA identical              PASS    max|Q_Q - Q_S| = 0.00e+00
Q-learning: optimal on a 3x3 open grid             PASS    4 steps vs optimal 4
SARSA: optimal on a 3x3 open grid                  PASS    4 steps vs optimal 4
Return accounting matches the environment          PASS    manual=-4446.0
-------------------------------------------------------------------------------
All 13 checks passed.
```

Appendix D

Per-Seed Results

The complete paired data underlying Chapter 7. Each row is one seed; both algorithms were trained under identical conditions for that seed, which is what licenses the paired tests.

The consistency is itself informative. Every single seed produced a 19-step route for Q-learning and a 33-step route for SARSA, which is why the bootstrap confidence interval on the route-length difference collapses to the degenerate interval $[14, 14]$.

	Q-learning			SARSA		
Seed	**Online**	**Incidents**	**Steps**	**Online**	**Incidents**	**Steps**
0	-849.6	22,107	19	-297.4	5,214	33
1	-838.6	21,717	19	-291.5	5,080	33
2	-904.3	23,362	19	-297.4	5,046	33
3	-734.6	18,752	21	-300.9	5,290	33
4	-768.2	19,843	19	-286.7	4,974	33
5	-857.9	22,127	19	-297.6	5,125	33
6	-744.6	19,117	19	-301.3	5,061	35
7	-884.0	22,882	19	-313.9	5,114	33
8	-1041.6	26,816	19	-302.0	4,887	33
9	-907.9	23,459	19	-291.1	5,042	33
10	-837.9	21,774	19	-294.7	5,250	33
11	-807.2	20,709	19	-301.5	5,046	33
12	-715.0	18,234	21	-292.8	5,073	33
13	-923.1	24,039	19	-289.4	4,904	33
14	-663.6	16,873	21	-307.2	5,186	35
15	-647.2	16,405	21	-298.9	5,026	33
16	-719.5	18,583	21	-289.8	5,123	33
17	-776.4	20,006	19	-308.9	5,025	33
18	-890.5	23,014	19	-287.8	5,000	33
19	-850.1	21,997	19	-302.0	5,019	35
20	-907.7	23,412	19	-288.2	5,012	33
21	-816.9	21,057	19	-302.8	5,255	33
22	-766.8	19,804	19	-303.5	5,203	33
23	-931.6	24,259	19	-287.6	5,001	33

	Q-learning			SARSA		
Seed	**Online**	**Incidents**	**Steps**	**Online**	**Incidents**	**Steps**
24	-655.7	16,918	21	-283.9	4,870	35
25	-935.8	24,115	19	-309.5	5,053	33
26	-1066.9	27,701	19	-301.1	5,112	33
27	-809.9	20,796	19	-289.9	5,057	nan
28	-1054.3	27,284	19	-296.9	4,956	35
29	-842.2	21,610	19	-303.2	5,031	33

Appendix E

Summary of Key Figures

Table E.1: Every headline quantity reported in this study.

Quantity	**Q-learning**	**SARSA**
Online return (mean over training)	−838.33	−297.32
Online return, final 20% of training	57.57	63.18
Walkway incidents during training	21,625.73	5,067.83
Deployed route length (steps)	19.40	33.34
Deployed success rate	1.00	0.97
Mean distance from the walkway	1.00	5.21
Excess steps over optimum, 12 floors	2.17	11.33
Incidents, 12-floor generality check	5,926.92	1,792.42

Table E.2: Shared experimental quantities.

Quantity	**Value**
Floor size	$20 \times 20 = 400$ states
Racking cells	67 (16.8%)
Optimal route (BFS)	19 steps
Seeds per algorithm	30, paired
Episodes per run	3,000
Verification checks passed	13/13
Generality check, Wilcoxon p	4.9×10^{-4}